Agent Ontology Practical - I

Dr. Kaneeka Vidanage

Imports and getting ready

- pip install mesa.
- pip install rdflib → For ontology traversal and inference

```
from mesa import Agent, Model
from mesa.time import RandomActivation
from rdflib import Graph, Literal, URIRef, Namespace
from rdflib.namespace import RDF, RDFS, OWL, XSD

# Define the namespace
NS = Namespace("http://example.org/booking.owl#")
```

Seating Arrangement

```python
class BookingRequestAgent(Agent):
    def __init__(self, unique_id, model):
        super().__init__(unique_id, model)
        self.requested_seat = None

    def step(self):
        # For demo purposes, we'll request seats in order
        seat_number = (self.unique_id - 1) % 10 + 1
        self.requested_seat = f"Seat{seat_number}"
        print(f"Agent {self.unique_id}: Requesting booking for {self.requested_seat}")
        self.model.booking_agent.process_request(self)
```

For example:

- If `self.unique_id` is 1, then `seat_number = (1 - 1) % 10 + 1 = 0 % 10 + 1 = 1`
- If `self.unique_id` is 12, then `seat_number = (12 - 1) % 10 + 1 = 11 % 10 + 1 = 1 + 1 = 2`

- This class is responsible for allow agents to make a request to book a seat.
- Step represents time-wise progression.

`seat_number = (self.unique_id - 1) % 10 + 1`

`self.unique_id`: This is a unique identifier for the agent. For example, if the agent's ID is 2, it means this is the second agent.

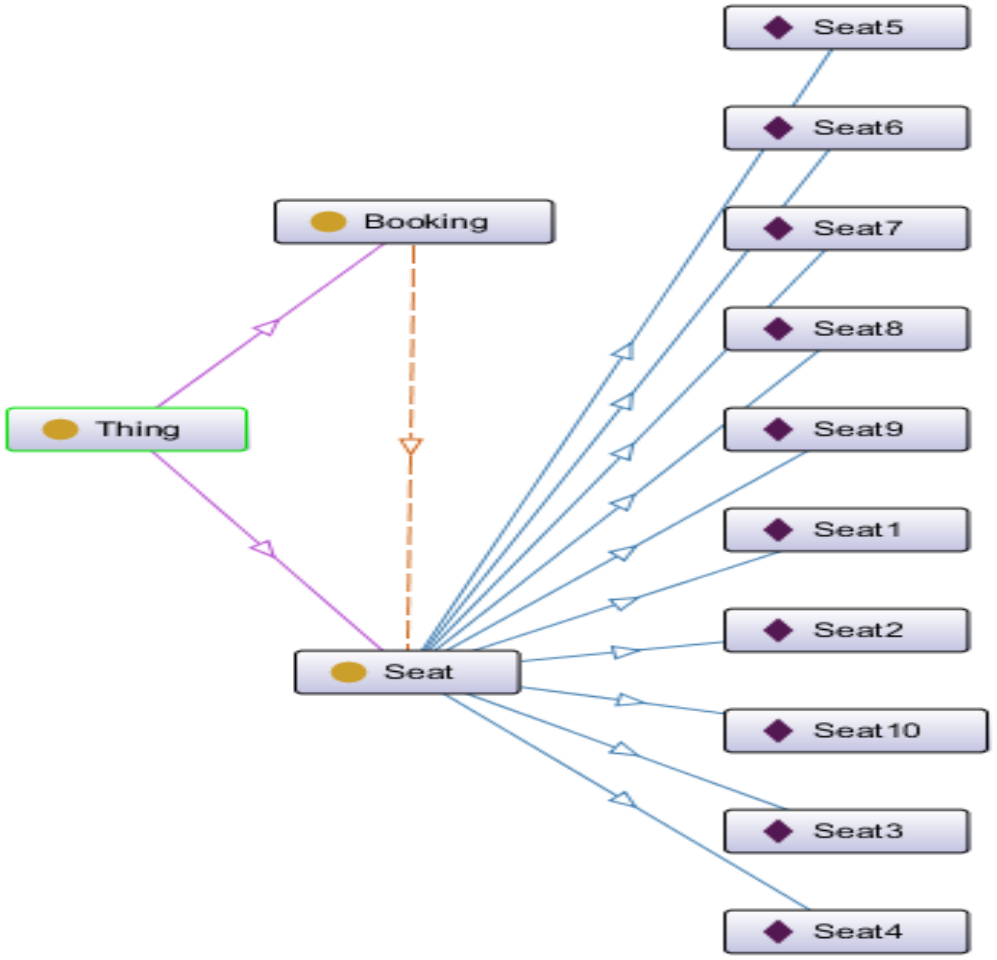
`self.unique_id - 1`: Subtracting 1 ensures that seat numbers start from 1 for the first agent instead of 0. This converts the agent ID into a zero-based index.

`% 10`: This operation ensures that the seat numbers wrap around when they exceed 10. For example, if the calculation results in 11, `% 10` makes it 1 again.

`+ 1`: Adjusts the zero-based index back to a 1-based seat number. Thus, seat numbers range from 1 to 10.

- This logic is used to prevent agents booking a seat greater than 10.

Booking Ontology in Protege



Booking a seat

```python
class BookingAgent(Agent):
    def __init__(self, unique_id, model):
        super().__init__(unique_id, model)
        self.graph = Graph()
        self.graph.parse("booking.owl", format="xml")  # Load the ontology

    def process_request(self, request_agent):
        seat_uri = NS[request_agent.requested_seat]

        # Check if the seat is available by traversing the graph
        for obj in self.graph.objects(seat_uri, NS.isAvailable):
            if obj == Literal(True, datatype=XSD.boolean):
                print(f"Agent {self.unique_id}: Booking confirmed for {request_agent.requested_seat}")
                # Mark the seat as unavailable after booking - updating rdf file
                self.graph.set((seat_uri, NS.isAvailable, Literal(False, datatype=XSD.boolean)))
                break
        else:
            print(f"Agent {self.unique_id}: {request_agent.requested_seat} is not available")
```

Execute Simulation

```python
class BookingModel(Model):
    def __init__(self):
        self.schedule = RandomActivation(self)

        self.booking_agent = BookingAgent(1, self)
        self.schedule.add(self.booking_agent)

        # Initialize agents with unique IDs
        self.request_agents = [BookingRequestAgent(i + 2, self) for i in range(10)]
        for agent in self.request_agents:
            self.schedule.add(agent)

    def step(self):
        self.schedule.step()
```

`BookingRequestAgent(i + 2, self)`:

- This part of the line constructs each `BookingRequestAgent` instance.
- `i`: Represents the current value in the range generated by `range(10)`. In this context, `i` takes on values from 0 to 9 (a total of 10 values).
- `i + 2`: Adds 2 to the current value of `i`. This is used to set the unique ID for each `BookingRequestAgent`. The IDs will be from 2 to 11.

Unique IDs: The purpose of `i + 2` is to generate a unique ID for each `BookingRequestAgent`, starting from 2 rather than 0. This ensures that the agent IDs are 2 through 11, rather than 0 through 9. This is often done to avoid potential conflicts or to align with specific requirements for IDs.

Class Definition:

- `BookingModel` inherits from `Model`, which is a base class in the Mesa framework. This means `BookingModel` follows the Mesa model structure and provides simulation behavior.

`__init__` Method:

- This method initializes the model when it is created.
- `self.schedule = RandomActivation(self)`:
  - Initializes a scheduler (`RandomActivation`) that manages the order in which agents are activated. `RandomActivation` ensures that agents are activated in random order at each step. This is useful for simulations where the order of agent actions is not fixed.
- `self.booking_agent = BookingAgent(1, self)`:
  - Creates an instance of the `BookingAgent` with a unique ID of 1 and adds it to the model's schedule. This agent handles the booking requests and updates seat availability.
- `self.schedule.add(self.booking_agent)`:
  - Adds the `BookingAgent` to the scheduler so that it will be activated during each step of the simulation.
- `self.request_agents = [BookingRequestAgent(i + 2, self) for i in range(10)]`:
  - Creates a list of 10 `BookingRequestAgent` instances with unique IDs ranging from 2 to 11. These agents simulate users who request seat bookings.
- `for agent in self.request_agents: self.schedule.add(agent)`:
  - Adds each `BookingRequestAgent` to the scheduler, ensuring they will also be activated during each step of the simulation.

Running the Simulation

```
# Run the model
if __name__ == '__main__':
    model = BookingModel()
    for _ in range(10):  # Run for a number of steps to test
        model.step()
```

Agent Interactions:

- Each `step` call advances the simulation by one time unit, allowing agents to perform their actions. Running the simulation for 10 steps means agents will have 10 opportunities to interact, request bookings, and process requests.

Testing and Debugging:

- **Short Simulations**: Running a simulation for a limited number of steps (like 10) is often useful for initial testing and debugging. It allows you to observe agent behaviors and interactions without waiting too long.
- **Longer Simulations**: To see long-term behavior or more complex interactions, you might need to run the simulation for more steps. This helps in understanding how agents' behavior evolves over time and how the system stabilizes.